

```
import pandas as pd

data = {
    'Brightness': [40, 50, 60, 10, 70, 60, 25],
    'Saturation': [20, 50, 90, 25, 70, 10, 80],
    'Class': ['Red', 'Blue', 'Blue', 'Red', 'Blue', 'Red', 'Blue']
}

df = pd.DataFrame(data)

df.to_csv("color_knn.csv", index=False)

print("Dataset created successfully!")
print(df)
```

Dataset created successfully!

	Brightness	Saturation	Class
0	40	20	Red
1	50	50	Blue
2	60	90	Blue
3	10	25	Red
4	70	70	Blue
5	60	10	Red
6	25	80	Blue

```
data = pd.read_csv("color_knn.csv")

print("Dataset:")
print(data)
```

Dataset:

	Brightness	Saturation	Class
0	40	20	Red
1	50	50	Blue
2	60	90	Blue
3	10	25	Red
4	70	70	Blue
5	60	10	Red
6	25	80	Blue

```
new_brightness = 55
new_saturation = 60

print("New Entry:")
print("Brightness:", new_brightness)
print("Saturation:", new_saturation)
```

New Entry:
Brightness: 55
Saturation: 60

```
import math

distances = []

for i in range(len(data)):

    brightness = data.loc[i, 'Brightness']
    saturation = data.loc[i, 'Saturation']

    distance = math.sqrt(
        (new_brightness - brightness) ** 2 +
        (new_saturation - saturation) ** 2
    )

    distances.append(distance)

data['Distance'] = distances

print("Distance of each point from the new entry:")
print(data)
```

Distance of each point from the new entry:

	Brightness	Saturation	Class	Distance
0	40	20	Red	42.720019
1	50	50	Blue	11.180340
2	60	90	Blue	30.413813
3	10	25	Red	57.008771
4	70	70	Blue	18.027756
5	60	10	Red	50.249378
6	25	80	Blue	36.055513

```
sorted_data = data.sort_values(by='Distance')

print("Data sorted according to distance:")
print(sorted_data)
```

```
Data sorted according to distance:
   Brightness  Saturation Class  Distance
1          50          50  Blue  11.180340
4          70          70  Blue  18.027756
2          60          90  Blue  30.413813
6          25          80  Blue  36.055513
0          40          20  Red   42.720019
5          60          10  Red   50.249378
3          10          25  Red   57.008771
```

```
k = 3

nearest_neighbors = sorted_data.head(k)

print("3 Nearest Neighbors:")
print(nearest_neighbors)
```

```
3 Nearest Neighbors:
   Brightness  Saturation Class  Distance
1          50          50  Blue  11.180340
4          70          70  Blue  18.027756
2          60          90  Blue  30.413813
```

```
class_counts = nearest_neighbors['Class'].value_counts()

print("Class Counts:")
print(class_counts)

predicted_class = class_counts.idxmax()

print("\nPredicted Class:", predicted_class)
```

```
Class Counts:
Class
Blue    3
Name: count, dtype: int64

Predicted Class: Blue
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics

X = data[['Brightness', 'Saturation']]
y = data['Class']

predictions = []

for i in range(len(data)):

    # Remove the current point
    X_train = X.drop(i)
    y_train = y.drop(i)

    # Current point for testing
    X_test = X.iloc[[i]]

    # KNN model
    model = KNeighborsClassifier(
        n_neighbors=3,
        metric='minkowski',
        p=2
    )

    model.fit(X_train, y_train)

    prediction = model.predict(X_test)

    predictions.append(prediction[0])

print("Actual Values:")
print(list(y))

print("\nPredicted Values:")
print(predictions)
```

```
Actual Values:
['Red', 'Blue', 'Blue', 'Red', 'Blue', 'Red', 'Blue']
```

```
Predicted Values:  
['Red', 'Blue', 'Blue', 'Red', 'Blue', 'Red', 'Blue']
```

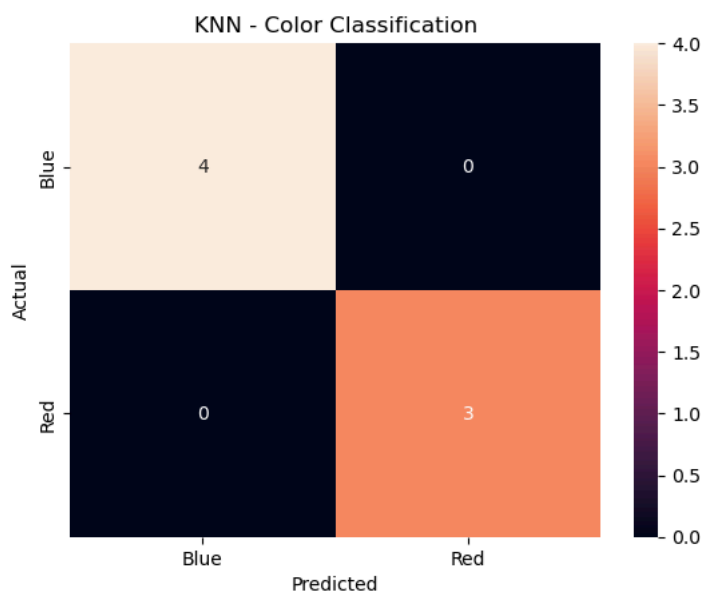
```
conf_mat = metrics.confusion_matrix(  
    y,  
    predictions,  
    labels=['Red', 'Blue']  
)  
  
print("Confusion Matrix:")  
print(conf_mat)
```

```
Confusion Matrix:  
[[3 0]  
 [0 4]]
```

```
accuracy_score = metrics.accuracy_score(  
    y,  
    predictions  
)  
  
print("Accuracy Score:", accuracy_score)  
print("Accuracy in Percentage:", int(accuracy_score * 100), "%")
```

```
Accuracy Score: 1.0  
Accuracy in Percentage: 100 %
```

```
import seaborn as sn  
import matplotlib.pyplot as plt  
  
conf_mat = pd.crosstab(  
    y,  
    predictions,  
    rownames=['Actual'],  
    colnames=['Predicted']  
)  
  
sn.heatmap(conf_mat, annot=True)  
  
plt.title("KNN - Color Classification")  
plt.show()
```



Start coding or [generate](#) with AI.

